

Traveling Salesman Problem Solved By Four Search Algorithms

MAHA MOHAMMED ALOSAIMI 44780314

The Traveling Salesman Problem (TSP) is a classic combinatorial optimization problem that seeks the shortest possible route that visits a given set of cities and returns to the origin city. Despite its simple premise, finding the optimal solution becomes computationally intractable as the number of cities increases. This project explores the application and performance of five distinct search algorithms for solving the TSP: Hill Climbing, Simulated Annealing, Genetic Algorithms, and Evolutionary Strategies, Branch and bound.

TSP main idea: You are a salesperson, and you have a list of cities you want to visit. You wish to travel to each city exactly once then return home. What order should you visit the cities to minimize the total distance you travel?

Despite its simple description, TSP is known for its computational complexity and has become a benchmark for evaluating optimization algorithms. TSP is not just a theoretical problem but has practical applications in various fields such as logistics, manufacturing, transportation, and even DNA sequencing.

Data preparation:

Dataset: In this project, we use two datasets. The first dataset used in this project is real data taken from the Ministry of Transportation [1]. As shown in the figure below. It contains the distance measurement in Kilometers between Saudi Arabia main cities such as(['Abha', 'Abqiq', 'Arar', 'Baha', 'Buraidah', 'Dammam', 'Hail', 'Hofuf', 'Hufr Al-Baten', 'Jazan', 'Jeddah', 'Kharj', 'Khobar', 'Madinah', 'Majma'ah', 'Makkah', 'Najran', 'Qurayat', 'Riyadh', 'Sakaka', 'Tabuk', 'Taif', 'Turaif'] [1].

المسافات الكيلومترية بين مدن المملكة (أقرب طريق) : 1434هـ

Distances Between Main Cities (Direct Routes) : 1434 A.H. (In Kilometers)

جدول 7 - 3

To	نجران	مكة المكرمة	عرعر	طريف	سكاكا	حفر الباطن	حائل	جازان	جدة	تبوك	بريدة	الهوف	المدينة المنورة	المجمعة	القرىات	الطائف	الرياض	الدمام	الخرج	الخبر	الباة	ابها	البحر	المن
From	Najran	Makkah	Arar	Turaif	Sakaka	Huf Al-Baten	Hail	Jazan	Jeddah	Tabuk	Buraidah	Hofuf	Madinah	Majma'ah	Qurayat	Taif	Riyadh	Dammam	Kharj	Khobar	Baha	Abha	Abqiy	المن
Abqiy	1282	1202	1470	1352	1288	545	972	1604	1281	1636	662	63	1180	517	1382	1114	332	87	319	63	1230	1262	0	البحر
Abha	257	574	2079	2239	925	1259	1618	229	674	1679	1378	1422	1007	1201	2083	498	1016	1432	938	1316	295	0	1262	ابها
Baha	552	279	1784	1944	1630	1468	1323	524	379	1384	1205	1391	712	1165	1788	203	985	1401	1072	1392	0	295	1230	الباة
Khobar	1689	1274	1071	1313	1225	506	1043	2108	1374	1698	778	124	1271	596	1469	1198	416	24	382	0	1393	1316	63	الخبر
Kharj	1195	936	1136	1378	1103	561	705	1323	1036	1336	440	337	933	258	1323	860	78	406	0	382	1072	938	319	الخرج
Dammam	1689	1274	1047	1289	1201	482	1043	1661	1374	1674	778	146	1271	596	1445	1198	416	0	406	24	1401	1432	87	الدمام
Riyadh	1273	858	1058	1300	1025	483	627	1245	958	1258	362	406	855	180	1245	782	0	416	78	416	985	1016	332	الرياض
Taif	755	76	1581	1741	1427	265	1120	727	176	1181	1002	1188	509	962	1585	0	782	1198	860	1198	203	498	1114	الطائف
Qurayat	2340	1509	398	156	368	1070	794	2231	1497	621	883	1651	1076	1373	0	1585	1245	1445	1323	1479	1788	2083	1382	القرىات
Majma'ah	1453	1038	878	1120	845	303	447	1425	1138	1078	182	586	675	0	1373	962	80	596	258	596	1165	1201	517	المجمعة
Madinah	1264	433	1072	1224	918	978	611	1155	421	672	493	1261	0	675	1076	509	855	1271	933	1471	712	1007	1180	المدينة المنورة
Hofuf	1679	1264	1159	1435	1431	629	1023	1651	1364	1664	786	0	1261	586	1651	1188	406	146	337	124	1391	1422	63	الهوف
Buraidah	1757	926	817	1039	663	485	265	1607	914	896	0	768	493	182	883	1002	362	778	440	778	1205	1378	662	بريدة
Tabuk	1936	1105	625	777	471	1381	657	1827	1093	0	896	1664	672	1078	621	1181	1258	1674	1336	1698	1384	1679	1636	تبوك
Jeddah	931	100	1493	1653	1339	1441	1032	734	0	1093	914	1364	421	1138	1497	176	958	1374	1016	1374	379	674	1281	جدة
Jazan	486	803	2227	2387	2073	1728	1766	0	734	1827	1607	1651	1155	1425	2231	727	1245	1661	1323	2108	524	229	1604	جازان
Hail	1875	1044	728	970	574	750	0	1766	1032	657	265	1033	611	447	794	1120	627	1043	705	1043	1323	1618	972	حائل
Huf Al-Baten	1756	1341	951	1193	1148	0	750	1728	1441	1381	485	629	978	303	1070	1265	483	482	561	506	1468	1259	545	حفر الباطن
Sakaka	2182	1351	154	396	0	1148	574	2073	1339	471	663	1431	918	918	368	1427	1025	1201	1103	1225	1630	1925	1288	سكاكا
Turaif	2496	1665	242	0	396	1193	970	2387	1653	777	1039	1435	1224	1120	156	741	1300	1289	1378	1313	1944	2239	1352	طريف
Arar	2231	1505	0	242	154	951	728	2227	1493	625	817	1159	1072	878	398	1581	1058	1047	1136	1071	1784	2079	1470	عرعر
Makkah	831	0	1505	1665	1351	1341	1044	803	100	1105	926	1264	433	1038	1509	76	858	1274	936	1274	279	574	1202	مكة المكرمة
Najran	0	831	2331	2496	2182	1756	1875	486	931	1936	1757	1679	1264	1453	2340	755	1273	1689	1195	1689	552	257	1282	نجران

Source : Ministry of Transportation

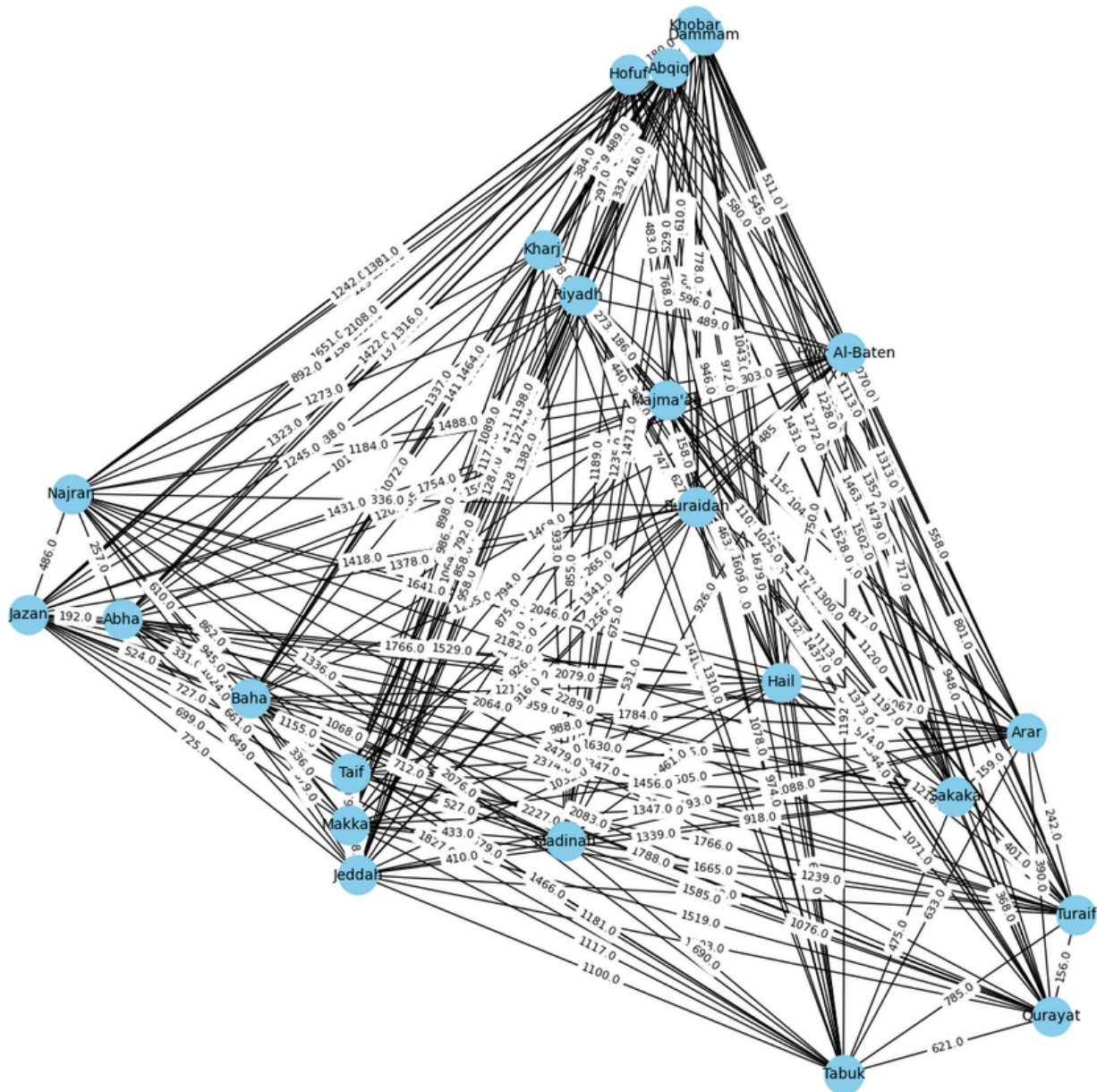
المصدر : وزارة النقل

The second dataset contains the latitude and longitude of Saudi Arabian cities to help with functions that need city coordinates [2].

city	lat	lng	country	iso2	admin_name	capital	population	population_proper
Riyadh	24.6500	46.7100	Saudi Arabia	SA	Ar Riyāḍ	primary	7237000	6694000
Jeddah	21.5428	39.1728	Saudi Arabia	SA	Makkah al Mukarr		4697000	4697000
Mecca	21.4225	39.8261	Saudi Arabia	SA	Makkah al Mukarr	admin	2427924	2427924
Ad Dammām	26.4333	50.1000	Saudi Arabia	SA	Ash Sharqīyah	admin	1532326	1386671
Medina	24.4700	39.6100	Saudi Arabia	SA	Al Madīnah al Mur	admin	1411599	1411599
Khamīs Mushayṭ	18.3000	42.7333	Saudi Arabia	SA	'Asīr		1353000	535065
Tabūk	28.3972	36.5789	Saudi Arabia	SA	Tabūk	admin	886036	886036
Buraydah	26.3333	43.9667	Saudi Arabia	SA	Al Qaṣīm	admin	467410	467410
Ḥafr al Bāṭin	28.4342	45.9636	Saudi Arabia	SA	Ash Sharqīyah		389993	389993
Al Jubayl	27.0000	49.6667	Saudi Arabia	SA	Ash Sharqīyah		378949	378949
Ḥā'il	27.5167	41.6833	Saudi Arabia	SA	Ḥā'il	admin	310897	310897
Yanbu'	24.0883	38.0582	Saudi Arabia	SA	Al Madīnah al Mur		298675	298675
Najrān	17.4917	44.1322	Saudi Arabia	SA	Najrān	admin	298288	298288
Al Kharj	24.1483	47.3050	Saudi Arabia	SA	Ar Riyāḍ		295462	295462
Al Qunfudhah	19.1264	41.0789	Saudi Arabia	SA	Makkah al Mukarr		272424	272424
'Unayzah	26.0840	43.9940	Saudi Arabia	SA	Al Qaṣīm		250000	250000
Sakākā	29.9697	40.2000	Saudi Arabia	SA	Al Jawf	admin	241669	150257
Abhā	18.2167	42.5000	Saudi Arabia	SA	'Asīr	admin	236157	236157
Muḥāyil	18.5464	42.0489	Saudi Arabia	SA	'Asīr		228979	228979
Ṣabyā	17.1490	42.6259	Saudi Arabia	SA	Jāzān		228375	228375
Al Khubar	26.3000	50.2167	Saudi Arabia	SA	Ash Sharqīyah		219679	219679
Ṣamītah	16.5972	42.9439	Saudi Arabia	SA	Jāzān		201656	201656
Al Qurayyāt	31.3333	37.3333	Saudi Arabia	SA	Al Jawf	minor	167080	167080
Al Ḥawīyah	21.4411	40.4975	Saudi Arabia	SA	Makkah al Mukarr		148151	148151

Next, we use networkx, which is a powerful open source Python library, we create a graph that has undirected edges. Undirected edges mean that the edge length from one node to the other is identical in both directions. This is also referred to as a symmetric TSP. here we use dataset2 latitude and longitude of Saudi Arabian cities as coordination to help create graph were each city node is in their approximately correct spot. and we use the distance between the cities from dataset1 as edge weight.

Saudi Cities Distance Graph (by Coordinates)



Implementation:

The code was evaluated using Google Colab. Visit the following link to view the whole implementation process

<https://colab.research.google.com/drive/1esXyWlhFE5AloLowgPjRXTfpUZAhyx7T?usp=sharing>

5 algorithms in total were used in this project implementation, where two of them are local search algorithms were evaluated in TSP, which are Hill Climbing and Simulated Annealing.

And the other two were metaheuristics algorithms: Genetic Algorithms and Evolutionary Strategies.

The fifth one is an informed search algorithm which is Branch and bound

Informed Search Algorithms

Branch and Bound

It uses a state-space tree for solving any problem. The algorithm works by recursively partitioning the search space into smaller subsets and pruning the subsets that cannot contain an optimal solution. Branch and Bound promise for an optimal solution [10], but it's not time efficient due to the factorial and exponential time complexity [10]. It needs to explore $(N-1)! / 2$ feasible solutions to determine the optimal solution [10].

Branch and Bound steps:

1. Start from the initial city and consider all possible next cities.
2. For each possible path, calculate a lower bound (an optimistic estimate of the minimum possible cost to complete the tour).
3. Use a priority queue always to expand the most promising (lowest bound) path first.
4. If a path's lower bound is worse than the best solution found so far, prune (ignore) it.
5. Continue expanding and pruning until all possibilities are explored or pruned.
6. The best complete path found is the optimal solution [14].

Local Search Algorithms

Stochastic Hill Climbing

It is a heuristic search algorithm that belongs to the family of local search methods. It is designed to solve problems where the goal is to find an optimal (or near-optimal) solution by iteratively moving from the current state to a better neighboring state [3]. The stochastic type introduces randomness into the search process. Instead of evaluating all neighbors or selecting the first improvement, it selects a random neighboring node and decides whether to move based on its improvement over the current state [3].

Stochastic Hill Climbing Steps :

1. Start with a random initial route (solution).
2. For a fixed number of iterations:
 - Generate a random neighbor by swapping two cities in the current route.
 - Calculate the total distance of the neighbor route.
 - If the neighbor is better (shorter distance), move to this neighbor (update current route).
 - Keep track of the best route found so far.
 - Record the best distance at each step for analysis.
3. Return the best route, its distance, and the history of best distances.

Key Tuned Variables:

- n_iterations: Number of iterations = 1000

Simulated Annealing

Simulated annealing is a probabilistic metaheuristic inspired by the annealing process in metallurgy [4]. It also starts with an initial solution and iteratively explores neighboring solutions. Unlike basic hill climbing, simulated annealing can accept worse solutions with a certain probability, which decreases over time (analogous to cooling). This allows it to escape local optima and explore the solution space more broadly, increasing the chance of finding a global optimum. The 'temperature' parameter controls the probability of accepting worse solutions [4].

Simulated Annealing Steps:

1. Start with a random initial route (solution).
2. Set the initial temperature.
3. For a fixed number of iterations:
 - Generate a random neighbor by swapping two cities in the current route.
 - Calculate the total distance of the neighbor route.
 - If the neighbor is better (shorter distance), move to this neighbor.
 - If the neighbor is worse, accept it with a probability based on the temperature and distance difference.
 - Update the best route found so far if the current route is better.
 - Decrease the temperature according to the cooling rate.
 - Record the best distance at each step for analysis.
4. Return the best route, its distance, and the history of best distances.

Key Tuned Variables:

initial_temp: The starting temperature. Controls the initial probability of accepting worse solutions. Higher values make the algorithm more likely to explore diverse solutions at the beginning

initial_temp = 1000

cooling_rate: The rate at which the temperature decreases. A value closer to 1 means slower cooling, allowing more exploration and a higher chance to escape local minima, but increases runtime. Lower values cool faster, making the search greedier

cooling_rate = 0.995

n_iterations :The total number of iterations. More iterations allow the algorithm to search the solution space more thoroughly, potentially finding better solutions, but increase computation time.

Metaheuristic Algorithms

Genetic Algorithms (GA)

Genetic algorithms are population-based metaheuristics inspired by natural selection and evolution. They maintain a population of candidate solutions (tours). In each generation, solutions are evaluated based on their fitness (shorter tours are fitter). Better solutions are more likely to be selected for reproduction, where they undergo crossover (combining parts of two parent tours to create offspring) and mutation (random changes to a tour). This process drives the population towards better solutions over generations [5].

Genetic Algorithm Steps:

1. Initialize a population of random routes (solutions).
2. For a fixed number of generations:
 - Evaluate the fitness (inverse of route distance) for each route.
 - Select the top-performing routes (elite) as parents.
 - Generate new children by crossover (combining two parents) and mutation (randomly swapping cities).
 - Form the new population from parents and children.
 - Track the best route and its distance in each generation.
3. Return the best route found, its distance, and the history of best distances.

Key Tuned Variables :

- `pop_size`: Number of routes in each generation. Larger values increase diversity but require more computation.

`pop_size = 100`

- `elite_size`: Number of top routes preserved unchanged. Higher values help retain good solutions but may reduce diversity.

`elite_size = 20`

Evolutionary Strategies (ES)

Evolutionary strategies are also population-based evolutionary algorithms, closely related to genetic algorithms. ES typically focuses more on mutation as the primary search operator, often employing self-adaptive mutation parameters. While GAs often use crossover to combine genetic material, ES might emphasize mutation to explore the search space. Both GAs and ES are powerful tools for complex optimization problems like the TSP, capable of finding near-optimal solutions for large instances [6].

Evolutionary Strategies (ES) Steps:

1. Initialize a population of random routes (solutions).
2. For a fixed number of iterations:
 - Evaluate the distance (fitness) of each route in the population.
 - Select the best route (with the shortest distance) as the parent for the next generation.
 - Generate a new population by copying the best route and applying mutation (randomly swapping two cities) to create diversity.
 - Track the best route and its distance in each iteration.
3. Return the best route found, its distance, and the history of best distances.

Key Tuned Variables:

- `population_size`: Number of routes in each generation. Larger values increase diversity but require more computation [6]

`population_size = 30`

- `n_iterations`: Number of iterations (generations). More iterations allow more improvement, but increase runtime.

`n_iterations = 500`

- `mutation_rate`: Probability of mutating a route (swapping two cities). Higher rates increase

- `mutation_rate`: Probability of mutating each route. Higher rates increase diversity and help escape local optima, but too high can disrupt good solutions.

`mutation_rate = 0.01`

- `generations`: Number of generations (iterations). More generations allow more improvement, but increase runtime.

`generation = 300`

Experimental Results And Comparison:

Comparison in terms of Optimal or completeness:

Algorithm	Optimal	Complete	Notes
Genetic Algorithm	No	No	Near-optimal, fast
Hill Climbing	No	No	May get stuck in local minima
Simulated Annealing	No	No	Good for escaping local minima
Evolutionary Strategy	No	No	Similar to GA, good for large n
Branch and Bound	Yes	Yes	Factorial and exponential time complexity

TSP is classified as an NP-hard problem, meaning that as the number of cities increases, the problem becomes exponentially harder to solve. That's why metaheuristic approaches for TSP are neither complete nor optimal.

Based on [7] [8] [9] [10] there is no exact algorithm is known to solve the TSP in polynomial time, heuristics and optimization-based methods such as Genetic algorithm, hill climbing, simulated annealing and evolutionary strategy algorithms are often employed to produce satisfactory, albeit sub-optimal, solutions. As the worst-case run time of any TSP algorithm can grow in a factorial time with the number of cities. That's why most research papers evaluate TSP algorithms based on the best or shortest path found for N cities of various sizes.

While [10] clarifies how informed search like Branch and bound can find the optimal solution for TSP. but it requires Factorial and exponential time complexity and it's not practical for large number of cities, cause it requires exploring $(n-1)!/2$ solutions to get to the optimal one. For me i had to reduce the number of cities from 23 to 12 only for this algorithm to reach to the optimal solution in efficient time.

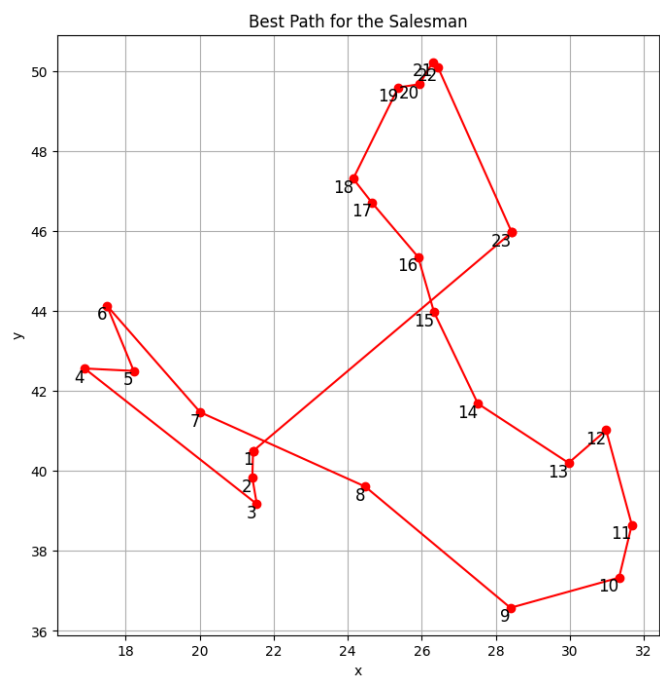
Comparison based on the shortest path found:

The following two tables show the best Solution for each selected algorithm, where the algorithm with the lowest best distance is considered the most effective for this dataset.

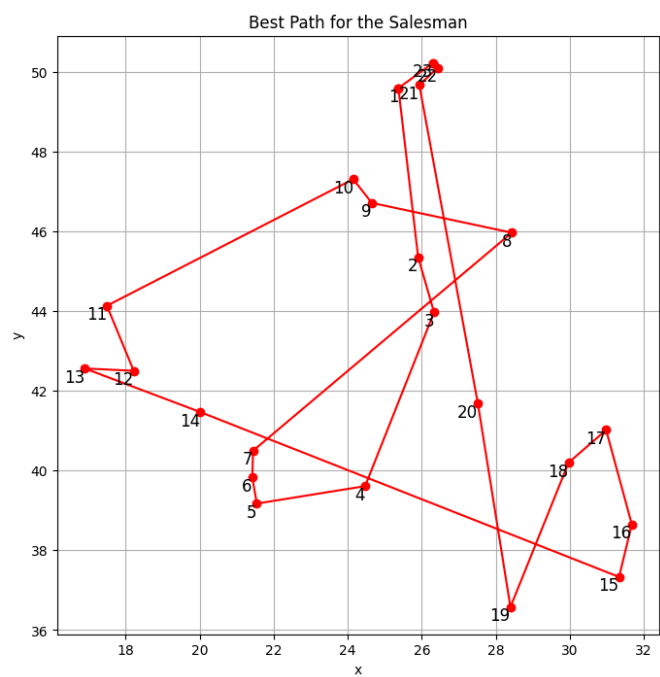
Algorithm

Best Path

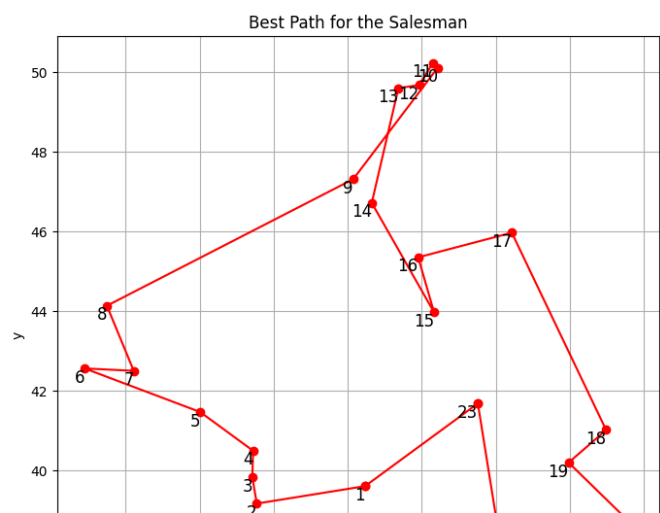
Genetic Algorithm

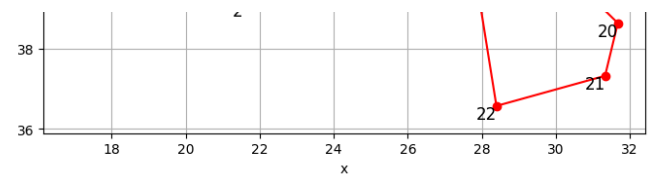


Hill-Climbing

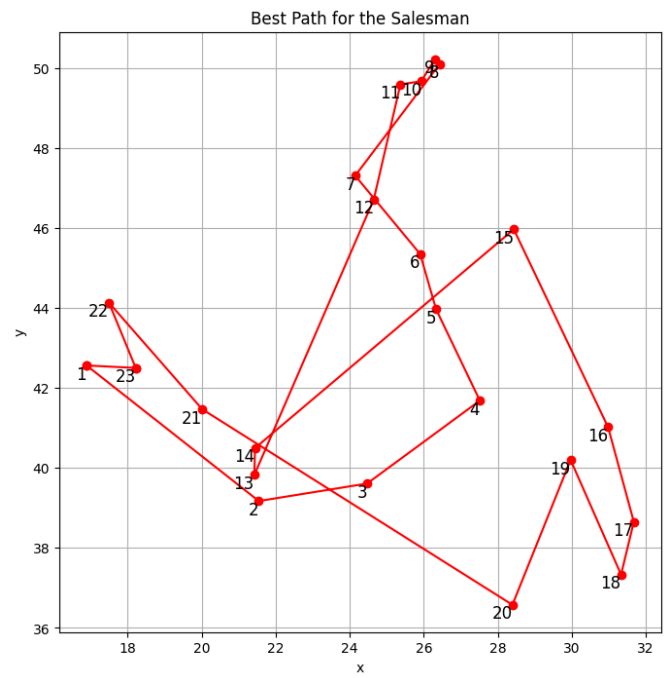


Simulated Annealing

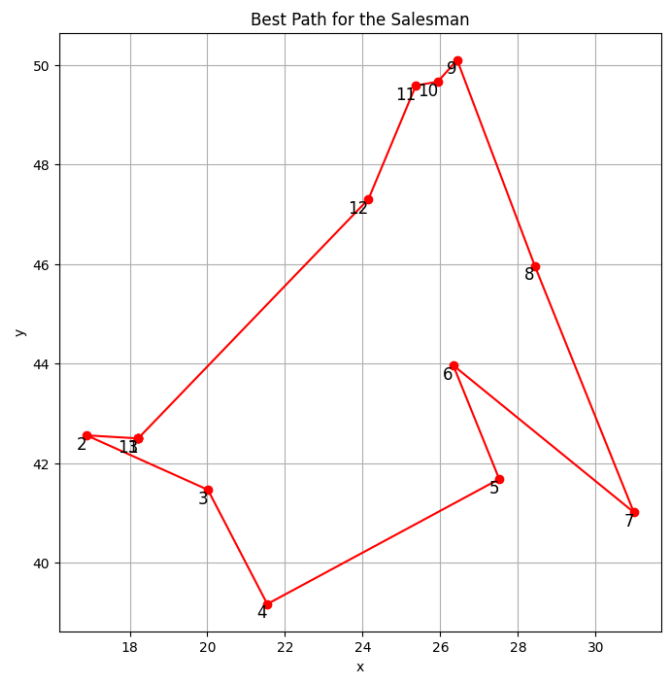




Evolutionary Strategies



Branch and Bound



Algorithm	Best Distance	Best Path
Genetic Algorithm	7103	Taif -> Makkah -> Jeddah -> Jazan -> Abha -> Najran -> Baha -> Madinah -> Tabuk -> Qurayat -> Turaif -> Arar -> Sakaka -> Hail -> Buraidah -> Majma'ah -> Riyadh -> Kharj -> Hofuf -> Abqiq -> Khobar -> Dammam -> Hufr Al-Baten -> Taif
Hill-Climbing	9162	Hofuf -> Majma'ah -> Buraidah -> Madinah -> Jeddah -> Makkah -> Taif -> Hufr Al-Baten -> Riyadh -> Kharj -> Najran -> Abha -> Jazan -> Baha -> Qurayat -> Turaif -> Arar -> Sakaka -> Tabuk -> Hail -> Abqiq -> Dammam -> Khobar -> Hofuf
Simulated Annealing	7362	Madinah -> Jeddah -> Makkah -> Taif -> Baha -> Jazan -> Abha -> Najran -> Kharj -> Dammam -> Khobar -> Abqiq -> Hofuf -> Riyadh -> Buraidah -> Majma'ah -> Hufr Al-Baten -> Arar -> Sakaka -> Turaif -> Qurayat -> Tabuk -> Hail -> Madinah
Evolutionary Strategies	8704	Jazan -> Jeddah -> Madinah -> Hail -> Buraidah -> Majma'ah -> Kharj -> Dammam -> Khobar -> Abqiq -> Hofuf -> Riyadh -> Makkah -> Taif -> Hufr Al-Baten -> Arar -> Turaif -> Qurayat -> Sakaka -> Tabuk -> Baha -> Najran -> Abha -> Jazan

Branch and Bound	5761	only for 12 cities: Abha -> Jazan -> Baha -> Jeddah -> Hail -> Buraidah -> Arar -> Hufu Al-Baten -> Dammam -> Abqiy -> Hofuf -> Kharj -> Abha
------------------	------	--

Analysis:

- **Genetic Algorithm** achieved the shortest route (**7103**), demonstrating its strong ability to explore the solution space and avoid local minima. This confirms its effectiveness for large and complex TSP instances.
- **Simulated Annealing** and **Evolutionary Strategies** both found good solutions, with best distances of **7362** and **8704**, respectively. These results are close to each other and indicate that both metaheuristic approaches are capable of finding high-quality solutions, though not as optimal as the Genetic Algorithm in this run.
- **Hill-Climbing** produced the longest route (**9162**), which is expected due to its tendency to get stuck in local minima and its lack of mechanisms to escape them.
- On the other hand, **Branch and Bound** has been evaluated only for 12 cities to generate the output in a realistic time, so we will not compare it with the other algorithms, where they run and search the path for all the cities in the dataset without any limitations. But branch and bound suppose to find the optimal solution trading of the time. [10] states that it was able to find the optimal solution.
- Overall the Genetic algorithm was more suitable for TSP than any other informed algorithms and local search algorithm and metaheuristic algorithms with outstanding performance with minimal times and semi-optimal path.

Conclusion

At the end, the Genetic Algorithm outperforms the other algorithms in term of the best shortest path found with **7103** km as distance needed to take a tour on Saudi Arabia and visit all the main cities once. despite that each of these five algorithms offers a different approach to tackling the Traveling Salesman Problem. Hill Climbing is the simplest but most prone to local optima. Simulated Annealing provides a probabilistic escape from local optima. Genetic Algorithms and Evolutionary Strategies, as population-based metaheuristics, offer more robust exploration of the solution space and are generally more effective for complex and large-scale TSP instances, though they typically require more computational resources and careful parameter tuning. The informed algorithm also takes factorial time but promises an optimal solution. But it's a bad choice for TSP since it becomes intractable for large input sizes.

resources

- [1] Kapsarc.org, 2026. <https://datasource.kapsarc.org/explore/assets/distances-between-main-cities-direct-routes-2009-in-kilometers/> (accessed May 24, 2026).
- [2] "Saudi Arabia Cities Database | Simplemaps.com," *Simplemaps.com*, 2026. <https://simplemaps.com/data/sa-cities> (accessed May 24, 2026).
- [3] GeeksforGeeks, "Hill Climbing in Artificial Intelligence," *GeeksforGeeks*, Dec. 12, 2017. <https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/>
- [4] GeeksforGeeks, "What is Simulated Annealing," *GeeksforGeeks*, Sep. 12, 2024. <https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/>
- [5] GeeksforGeeks, "Genetic Algorithms," *GeeksforGeeks*, Jun. 29, 2017. <https://www.geeksforgeeks.org/dsa/genetic-algorithms/>
- [6] pawelkauf, "Introduction to Evolutionary Strategies 🧬," *Kaggle.com*, Jan. 22, 2025. <https://www.kaggle.com/code/pawelkauf/introduction-to-evolutionary-strategies> (accessed May 24, 2026).
- [7] Y. Khdeir and A. Awad, "A Comparison of Heuristic Algorithms for Solving the Traveling Salesman Problem," *An-Najah University Journal for Research - A*, vol. 39, no. 1, pp. 73–80, Feb. 2025, doi: [10.35552/anujr.a.39.1.2301](https://doi.org/10.35552/anujr.a.39.1.2301).
- [8] K. Karagul, E. Aydemir, and S. Tokat, "Using 2-Opt based evolution strategy for travelling salesman problem," *Int. J. Optim. Control, Theor. Appl. (IJOCTA)*, vol. 6, no. 2, pp. 103–113, Mar. 2016, doi: [10.11121/ijocta.01.2016.00268](https://doi.org/10.11121/ijocta.01.2016.00268).
- [9] T. Babu, R. R. Nair, S. Sumanth, I. T. P, and C. K. S, "Hybrid Approach: Combining Hill Climbing and Genetic Algorithms for Traveling Salesman Problem," in *2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD)*, Manama, Bahrain: IEEE, Apr. 2025, pp. 1–5. doi: [10.1109/ITIKD63574.2025.11004701](https://doi.org/10.1109/ITIKD63574.2025.11004701).
- [10] CHRISTINA V. STERGAARD, *CLOSER LOOK AT THE TRAVELLING SALESMAN PROBLEM*. in Mathematics Research Developments Ser. NEW YORK: NOVA SCIENCE, 2020.
- [11] "Simulated Annealing TSP : Parameterising Simulated Annealing for the Travelling Salesman Problem  | Kaggle," *Kaggle.com*, 2026. <https://www.kaggle.com/discussions/getting-started/285842> (accessed May 24, 2026).
- [12] J. Wilkins, "Solving The Travelling Salesman Problem Using A Genetic Algorithm | Towards Data Science," *Towards Data Science*, Aug. 25, 2024. <https://towardsdatascience.com/solving-the-travelling-salesman-problem-using-a-genetic-algorithm-c3e87f37f1de/>
- [13] E. Howell, "How To Solve Travelling Salesman Problem With Simulated Annealing | Towards Data Science," *Towards Data Science*, Feb. 10, 2023. <https://towardsdatascience.com/how-to-solve-travelling-salesman-problem-with-simulated-annealing-c248447a8bcd/>

[14] O. Yenigün, "Traveling Salesman Problem: Branch and Bound Solution," Medium. Accessed: May 26, 2026. [Online]. Available: <https://towardsdev.com/traveling-salesman-problem-branch-and-bound-solution-c042d1a2c36d>